Optical Flow Assignment, Mathematical Modeling 02526 S26

William Michelsen (wimi@itu.dk) Maciej Jałocha (macja@itu.dk)
(Equal contributions)

12.03.2026

1 Introduction

Optical flow is a fundamental computer vision technique for estimating the apparent motion of objects in a video sequence by calculating pixel displacements between frames. Mathematical approaches such as the Lucas–Kanade method provide a clear framework based on brightness constancy and local spatial coherence, but their strict assumptions can conflict with real-world conditions. To study these limitations, we implemented a dense optical flow estimator using Lucas–Kanade with 3D Gaussian pre-filtering. We then evaluated the implementation on custom smartphone recordings, including rolling-ball and aperture-problem scenarios designed to either satisfy or violate the model assumptions. This report presents the relevant mathematical formulation, describes the data collection process, and visualizes the estimated motion fields to assess the practical usefulness of the Lucas–Kanade method.

2 Data

The data used to study the limitations of optical flow were self-collected. We recorded the examples with our smartphones and converted the frames to grayscale. Details are provided in Table 1.

Table 1: Overview of recorded examples and acquisition settings.

Description	Example Type	Recording device details
Aperture Problem	Adversarial	OnePlus Nord 3, 30 fps, 1080p
Rolling Ball	Friendly	OnePlus Nord 3, 30 fps, 1080p
Soap	Friendly	iPhone 16 Pro, 60 fps, 4K
Fast	Adversarial	iPhone 16 Pro, 60 fps, 4K

Descriptions of the examples:

- The Aperture Problem example shows a piece of plastic pulled rightward while appearing to move upward in the image. We expect the model to fail to capture the true direction.
- The Rolling Ball example shows a dark ball rolling slowly after being pushed, filmed from above. We expect this case to be easy for the model.
- The Soap example shows a colorful package of solid soap in motion. We expect the model to capture the motion well because of the rich texture and slow movement.

- The Fast example shows an untextured box moving rapidly. We expect the speed and lack of texture to challenge the model.

3 Methods

In this study, we focus on optical flow estimation using Gaussian filters and the Lucas–Kanade method.

3.1 Optical Flow

The optical flow equation in continuous form is:

$$V_x(p)\Delta x + V_y(p)\Delta y = -V_t(p) \tag{1}$$

Here, the volume V returns brightness at spatial position (x, y) and time t, so V takes three real-valued variables (x, y, t). The terms $V_x = \frac{\partial V}{\partial x}$, $V_y = \frac{\partial V}{\partial y}$, and $V_t = \frac{\partial V}{\partial t}$ are partial derivatives of V with respect to space and time. $p = [x, y, t]^T$ denotes position, and $u = [\Delta x, \Delta y, 1]^T$ denotes displacement over one frame ($\Delta t = 1$). We assume that an object occupies a single point at each instant.

The equation holds under two assumptions. The first is brightness constancy: $V(p + u) - V(p) = 0$. This means that an object keeps constant brightness as it moves. The second is that the displacement u is small. A derivation is given in Appendix A.1.

To obtain Δx and Δy for an object currently at position (x, y) and time t, we need to (a) estimate $V_x(p)$, $V_y(p)$, and $V_t(p)$, and (b) solve the flow equation. For a single pixel, the equation is underdetermined: there is one equation and two unknowns, Δx and Δy.

Next, we present how to solve the optical flow equation in a discrete setting.

3.1.1 Estimating volume partial derivatives using filters

We estimate volume partial derivatives using derivatives of three-dimensional filters. For each differentiation dimension, the corresponding filter includes smoothing across the remaining two dimensions. A schematic visualization of such filters is shown in Figure 3.

We specifically use Gaussian filters [2]. Compared with Sobel or Prewitt filters, they allow direct control over smoothing.

For all three derivative filters, the shape across dimensions (x, y, t) is $(5, 5, 3)$ and the per-dimension standard deviations are $(1, 1, 1)$. When part of a filter extends beyond the image boundary, we extend the input by reflection at the edge pixel. In toy examples, this reduced irrelevant optical flow vectors near image boundaries compared with zero padding.

3.1.2 Solving the underdetermined optical flow equation with Lucas–Kanade

The Lucas–Kanade method [1] solves the optical flow equation (1) by incorporating neighboring pixels and solving the resulting system with least squares, as shown in Equation (2):

$$\begin{bmatrix} V_x(p_1) & V_y(p_1) \\ \vdots & \vdots \\ V_x(p_n) & V_y(p_n) \end{bmatrix} \begin{bmatrix} \Delta x \\ \Delta y \end{bmatrix} = - \begin{bmatrix} V_t(p_1) \\ \vdots \\ V_t(p_n) \end{bmatrix} \tag{2}$$

The simplest implementation iterates over pixels in a frame and solves the equation for each pixel neighborhood.

Our approach differs from the simplest version in two ways. First, we assign lower weights to neighbors (x, y, t) that are farther from p, which helps reduce noise from positions that may belong to unrelated objects. Second, we precompute volume-level terms using Gaussian filters, which reduces computational cost.

$$\begin{bmatrix} S_{xx} & S_{xy} \\ S_{xy} & S_{yy} \end{bmatrix} \begin{bmatrix} \mathbf{\Delta x} \\ \mathbf{\Delta y} \end{bmatrix} = - \begin{bmatrix} S_{xt} \\ S_{yt} \end{bmatrix} \tag{3}$$

Figure 1: Lucas–Kanade normal equation. $\mathbf{\Delta x}$ and $\mathbf{\Delta y}$ contain optical flow vectors for the volume. S_{ab} is an element-wise product of partial derivatives of the volumes V_a and V_b after per-pixel filtering, i.e., $S_{ab} = G * (V_a \odot V_b)$.

Specifically, we use a Gaussian filter, which also performs the weighting across all three dimensions.

3.2 Visualizing optical flow

We visualize our experiments using quiver plots. We filter out vectors whose magnitudes are below either a per-frame Otsu threshold [3] or a per-volume noise threshold. The noise threshold is defined as the 99th percentile of all magnitudes in a selected static part of the volume. We scale vectors by 1:10.

We used the toy example to experimentally assess whether this was an informative presentation of the results (Figure 4).

4 Results

Aperture Problem and Rolling Ball results in Figure 2 surprised us.

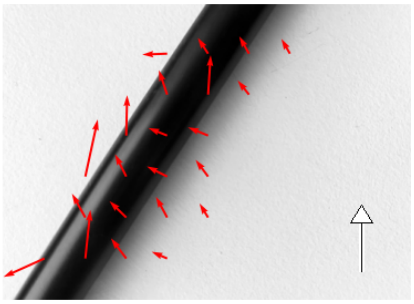
We expected the Aperture Problem 2a to break optical flow estimation. We expected vectors to point only upward, but we observed a few pointing in the true direction and many pointing diagonally.

We also expected the Rolling Ball 2b scenario to be favorable. However, many vectors pointed in irrelevant directions and had larger-than-expected magnitudes.

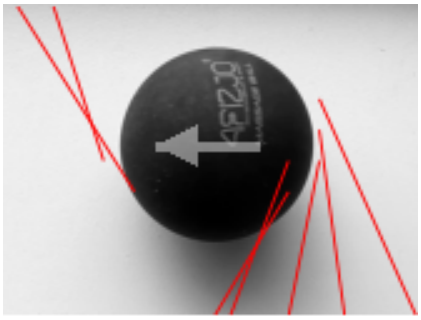
The remaining two problems, Fast 2c and Soap 2d followed our expectations.

5 Discussion

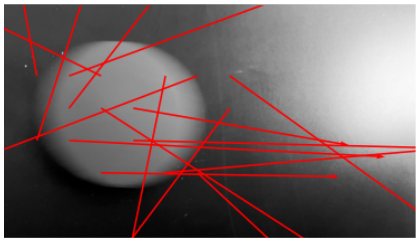
Our results showed several unexpected behaviors, particularly for the Rolling Ball and Aperture Problem sequences. Although we designed the Rolling Ball video (Figure 2b) to be an ideal case with constant lighting and good texture, the estimated optical flow vectors were large and often pointed in incorrect directions. This likely happened because the Lucas–Kanade model assumes that a local patch moves via simple 2D translation. A rolling ball rotates in 3D, so surface texture warps and changes perspective from frame to frame, violating brightness constancy. Additionally, because the ball was pushed manually, it may have moved too many pixels between frames, violating



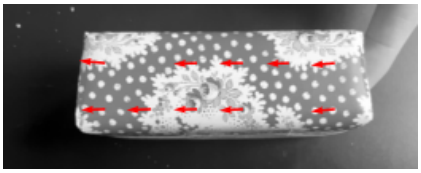
(a) Aperture problem example with estimated optical flow vectors (red arrows). Frame 78. The vectors are taken every 50th pixel.



(b) Rolling ball example with estimated optical flow vectors (red arrows). Frame 60. The vectors are taken every 20th pixel. White arrow indicates the ball's movement direction.



(c) Fast-motion example with estimated optical flow vectors, taken every 50th pixel.



(d) Soap example with estimated optical flow vectors, taken every 50th pixel.

Figure 2: Qualitative optical flow results across the four experimental scenarios. Optical flow vectors are scaled 1:10. The vectors are filtered by magnitude using a per-volume 99th percentile background noise threshold and a per-frame Otsu threshold.

the small-displacement assumption required by the model. Finally, the ball's texture may have been too uniform, limiting the model's ability to detect reliable motion.

On the other hand, the Aperture Problem sequence (Figure 2a) performed better than expected. We pulled a relatively untextured piece of plastic sideways and expected the model to detect only motion perpendicular to the edge. While many vectors pointed diagonally, a substantial portion correctly pointed rightward. One explanation is that the model captured motion cues from light reflections. Another explanation may come from our implementation: Gaussian weighting of neighboring pixels. By smoothly aggregating information over a wider area while downweighting distant pixels, the filter may have captured subtle gradients from microscopic surface texture, dust particles, or object corners entering the frame. These details can provide enough gradient variation for the least-squares system to recover the true motion direction.

The Fast and Soap examples, however, performed as expected. In the Fast example, the motion was too rapid, most likely violating the assumption of small displacement u. In the Soap example, slow motion and diverse texture helped satisfy the optical flow assumptions and produced vectors with correct directions.

These findings highlight how sensitive the Lucas–Kanade method is to real-world physics. The model works well for controlled 2D motion, but it struggles with unconstrained 3D rotation and fast motion, both of which are common in handheld smartphone recordings.

To make tracking more reliable in future work, we could reduce inter-frame displacement by record-

ing at a higher frame rate or downscaling video resolution. Another option is to estimate flow only at sharp corners or highly textured regions instead of computing dense flow across flat, ambiguous surfaces.

6 Conclusions

Our analysis demonstrates that our Lucas–Kanade optical flow implementation is fundamentally constrained by strict mathematical assumptions, making it highly sensitive to real video footage. We found that spatial weighting via 3D Gaussian filters can provide unexpected robustness in seemingly textureless environments by aggregating weak gradients, but the core assumption of constant brightness under 2D translation breaks down for objects undergoing 3D rotation. Overall, standard dense optical flow appears unreliable for raw everyday smartphone footage, highlighting the need for smaller displacements or more selective tracking in practical applications.

References

[1] B. D. Lucas and T. Kanade, *An Iterative Image Registration Technique with an Application to Stereo Vision*, Proceedings of the 7th International Joint Conference on Artificial Intelligence (IJCAI), pp. 674–679, 1981.

[2] P. Virtanen et al., *SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python*, Nature Methods, 17:261–272, 2020.

[3] N. Otsu, *A Threshold Selection Method from Gray-Level Histograms*, IEEE Transactions on Systems, Man, and Cybernetics, 9(1):62–66, 1979.

A Appendix

A.1 Optical Flow Equation derivation

Proof. Assume the notation introduced in the main text. Under brightness constancy, $V(p+u) = V(p)$. A first-order Taylor expansion gives

$$V(p+u) = V(p) + u \cdot V'(p) + O(\|u\|^2).$$

Using the small-displacement assumption, we neglect higher-order terms and obtain

$$V(p+u) \approx V(p) + u \cdot V'(p).$$

Substituting into brightness constancy yields

$$u \cdot V'(p) = 0,$$

which expands to

$$\Delta x V_x + \Delta y V_y + V_t = 0.$$

Rearranging gives

$$\Delta x V_x + \Delta y V_y = -V_t.$$

□

A.2 Figures

A.3 Use of Generative AI

The analysis code was written almost entirely by us, with GitHub Copilot used for debugging. The report was written by us; we used OpenAI's Prism for proofreading, formatting, and figure creation.

A.4 Analysis code

```
# %% [markdown]
# ### Initial toy problem

# %%
import matplotlib
```

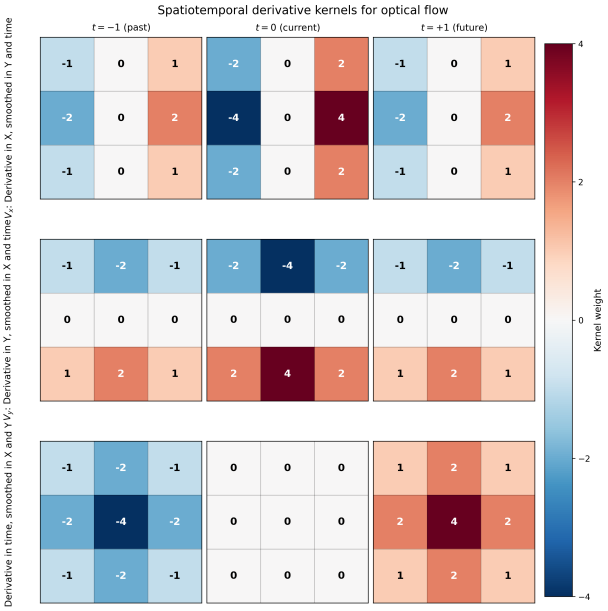


Figure 3: **Solely schematic figure:** Spatiotemporal derivative $3 \times 3 \times 3$ filters for optical flow: rows are V_x, V_y, and V_t; columns are time slices $t = -1$, $t = 0$, and $t = +1$. The filter moves over each pixel, performs an element-wise multiplication with the pixel neighborhood, and computes a weighted average.

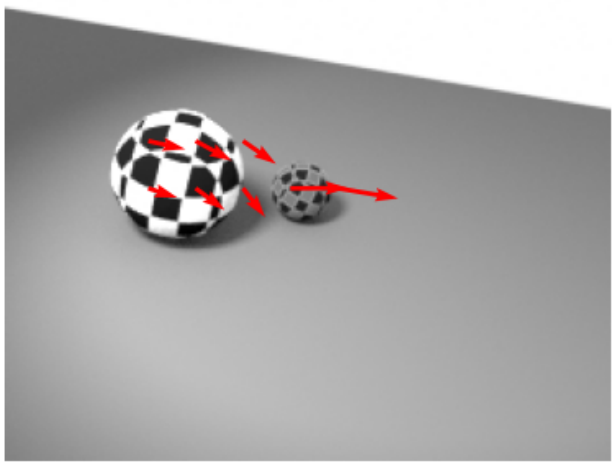


Figure 4: Toy example scene used for qualitative tuning of optical flow parameters. Red arrows represent optical flow vectors, scaled 1:10. Plotted every 20 pixels. Filtered with a 99th-percentile volume background-noise threshold and an Otsu per-frame threshold.

```
import numpy as np
import os
import matplotlib.pyplot as plt
import matplotlib

path = "./toy_problem/"


# load all the images inside into 3d numpy array
def load_images(path):
    images = []
    for filename in sorted(os.listdir(path)):
        if filename.endswith(".png"):
            img = plt.imread(os.path.join(path, filename))
            # convert to grayscale
            img = np.mean(img, axis=2, dtype=np.float32) / 255.0
            images.append(img)
    return np.array(images)


# Load images
frames = load_images(path)
print(frames.shape)


# %%
```

```
# Animate the images using matplotlib and pause
plt.figure()
for i in range(frames.shape[0]):
    plt.imshow(frames[i], cmap="gray")
    plt.title(f"Frame {i + 1}")
    plt.axis("off")
    plt.pause(0.1)
plt.show()

# %%


# %% [markdown]
# ### Problem 2.1 - Low level gradient calculation: The most simple way is to approximate the gradient
# as the difference between neighboring pixels/voxels.
# In a 1D signal, x, this would be:
# MATLAB: dx = x(2:end) - x(1:end-1)
# PYTHON: dx = x[1:] - x[0:-1]
# Obtain Vx,Vy,Vt for the entire volume and display for some selected frames.
# Consider what happens with the image/volume size, when performing gradient calculations like this?

# %%
frames.shape

# %%
V = frames[1:, 1:, 1:] - frames[:-1, :-1, :-1]

# frame = V[50, :, :]
Vx_raw = frames[:, :, 1:] - frames[:, :, :-1]
Vy_raw = frames[:, 1:, :] - frames[:, :-1, :]
Vt_raw = frames[1:, :, :] - frames[:-1, :, :]

plt.figure()
plt.imshow(Vx_raw[50], cmap="gray")
plt.title("Optical Flow (Frame 1)")
plt.axis("off")
plt.show()

# %%
frames[50].min()

# %%
frame.min()

# %% [markdown]
# When performing such a gradient the dimensions dimnishes, definitely.
# * Image size - decreases
# * Volume size - well, you substract stuff, so that also gets smaller.
# * You actually end up with a range [-1,1] or [`-frame.max()`, `frame.max()`]

# %% [markdown]
# ### Problem 2.2 - exploring filters

# %%
import scipy

fig, ax = plt.subplots(nrows=1, ncols=4, figsize=(15, 5))
```

```
ax[0].imshow(
    scipy.ndimage.sobel(frames[50], mode="constant", cval=0.0, axis=0),
    cmap="gray",
)
ax[0].set_title("Sobel Filter (Axis 0)")

ax[1].imshow(
    scipy.ndimage.sobel(frames[50], mode="constant", cval=0.0, axis=1),
    cmap="gray",
)
ax[1].set_title("Sobel Filter (Axis 1)")

ax[2].imshow(
    scipy.ndimage.sobel(frames, mode="constant", cval=0.0, axis=0)[50],
    cmap="gray",
)
ax[2].set_title("Sobel Filter around time")

ax[3].imshow(frames[50], cmap="gray")
ax[3].set_title("Original Frame")
plt.show()

# %%
frames.shape

# %%


# %%


# %%
plt.imshow(frames[])

# %%
plt.imshow(frames[49] - frames[51], cmap="gray")

# %%
# flipped sobel
plt.imshow(
    -scipy.ndimage.sobel(frames[50], mode="constant", cval=0.0, axis=0),
    cmap="gray",
)

# %% [markdown]
# Conclusions:
# * axis=0, means horizontal kernel (alongside rows)
# * axis=1 means vertical kernel (alongside columns)
# * batching does not work (cannot do this with 3 at the same time haha, at least when the time is first)
# * Flipping - for kernels like sobel and prewit, it's simply about taking a negative.
# * To actually understand the underlying implementation - I would need to spend some time on the underlying logic.

# %% [markdown]
# ### Problem 2.3 - exploring multidimensional gaussian filter

# %%
# gaussian filter as derivative
```

```

fig, ax = plt.subplots(nrows=1, ncols=4, figsize=(15, 5))
ax[0].imshow(
    scipy.ndimage.gaussian_filter(frames, sigma=0.5, order=[0, 0, 1])[50],
    cmap="gray",
)
ax[0].set_title("Gaussian Filter Vx (Axis 2)")

ax[1].imshow(
    scipy.ndimage.gaussian_filter(
        frames, sigma=0.5, order=[0, 1, 0], mode="constant", cval=0.0
    )[50],
    cmap="gray",
)
ax[1].set_title("Gaussian Filter Vy (Axis 1)")

ax[2].imshow(
    scipy.ndimage.gaussian_filter(
        frames, sigma=1, order=[1, 0, 0], mode="constant", cval=0.0
    )[50],
    cmap="gray",
)
ax[2].set_title("Gaussian Filter Vt (Axis 0 / time)")

ax[3].imshow(frames[50], cmap="gray")
ax[3].set_title("Original Frame")
plt.show()

# %%


# %%
tmp[:50, :] = 0
plt.figure()
plt.imshow(tmp, vmin=0, vmax=0.01, cmap="gray")
plt.show()

# %%


# %%
tmp = frames[50].copy()

# %% [markdown]
# ### 3.1 - Lukas Kanade local level solution

# %%
Vx = scipy.ndimage.gaussian_filter(frames, sigma=1, order=[0, 0, 1])
Vy = scipy.ndimage.gaussian_filter(frames, sigma=1, order=[0, 1, 0])
Vt = scipy.ndimage.gaussian_filter(frames, sigma=1, order=[1, 0, 0])

# %%
# Simple:

t = 30
x, y = (100, 150)

N = 5

```

```
205
206 def get_region_around(x, y, N, arr):
207     """Region centered around (x, y) with size NxN"""
208     return arr[y - N : y + N + 1, x - N : x + N + 1]
209
210
211 Vx_region = get_region_around(x, y, N, Vx[t])
212 Vy_region = get_region_around(x, y, N, Vy[t])
213 Vt_region = get_region_around(x, y, N, Vt[t])
214
215 A = np.hstack((Vx_region.reshape(-1, 1), Vy_region.reshape(-1, 1)))
216 b = -Vt_region.reshape(-1, 1)
217 # Solve for the flow vector (u, v) using least squares
218 flow_vector, _, _, _ = np.linalg.lstsq(A, b)
219 # print("Estimated flow vector (u, v):", flow_vector
220
221 # %%
222 # Plot the displacement vector on the original frame
223 plt.figure()
224 plt.imshow(frames[t], cmap="gray")
225 scaled_flow_vector = flow_vector.flatten() * 10 # Scale for better visualization
226 plt.quiver(
227     x,
228     y,
229     scaled_flow_vector[0],
230     scaled_flow_vector[1],
231     color="red",
232     angles="xy",
233     scale_units="xy",
234     scale=1,
235 )
236 plt.title("Estimated Flow Vector")
237 plt.axis("off")
238 plt.show()
239
240 # %%
241
242
243 # %% [markdown]
244 # ### To discuss
245 # Edge strategy
246 # I would apply this normalisation strategy - utilise only the data that is available (as the guy
        said in the video).
247 # However, for now - let me go fast, and talk with William afterwards on how to do it.
248
249 # %%
250 A.shape
251
252 # %% [markdown]
253 # ### 3.2 - APply LK to full Volume + Visualisation
254 # APply to full volume
255
256 # %%
257 # test 3d array
258 from pylab import dtype
259
260 arr = np.random.rand(3, 3, 3)
261 arr
262
```

```
# %%
# go over all pixels in the frame, with a stride of 5, and compute the flow vector for each selected pixel and plot
t = 16
N = 11
stride = 20
plt.figure(figsize=(10, 10))
plt.imshow(frames[t], cmap="gray")
for y in range(N, frames[t].shape[0] - N, stride):
    for x in range(N, frames[t].shape[1] - N, stride):
        Vx_region = get_region_around(x, y, N, Vx[t])
        Vy_region = get_region_around(x, y, N, Vy[t])
        Vt_region = get_region_around(x, y, N, Vt[t])

        A = np.hstack((Vx_region.reshape(-1, 1), Vy_region.reshape(-1, 1)))
        b = -Vt_region.reshape(-1, 1)
        flow_vector, _, _, _ = np.linalg.lstsq(A, b)
        scaled_flow_vector = flow_vector.flatten() * 10
        plt.quiver(
            x,
            y,
            scaled_flow_vector[0],
            scaled_flow_vector[1],
            color="red",
            angles="xy",
            scale_units="xy",
            scale=1,
        )
plt.title("Estimated Flow Vectors")
plt.axis("off")
plt.show()

# %%
# repeat for all frames (output all 64 images with flow vectors)

for t in range(frames.shape[0]):
    N = 3
    stride = 20
    plt.figure(figsize=(10, 10))
    plt.imshow(frames[t], cmap="gray")
    for y in range(N, frames[t].shape[0] - N, stride):
        for x in range(N, frames[t].shape[1] - N, stride):
            Vx_region = get_region_around(x, y, N, Vx[t])
            Vy_region = get_region_around(x, y, N, Vy[t])
            Vt_region = get_region_around(x, y, N, Vt[t])

            A = np.hstack((Vx_region.reshape(-1, 1), Vy_region.reshape(-1, 1)))
            b = -Vt_region.reshape(-1, 1)
            flow_vector, _, _, _ = np.linalg.lstsq(A, b)
            scaled_flow_vector = flow_vector.flatten() * 10
            plt.quiver(
                x,
                y,
                scaled_flow_vector[0],
                scaled_flow_vector[1],
                color="red",
                angles="xy",
                scale_units="xy",
                scale=1,
```

```
            )
    plt.title(f"Estimated Flow Vectors (Frame {t + 1})")
    plt.axis("off")
    plt.show()

# %%
# repeat for all frames (output all 64 images with flow vectors)

for t in range(frames.shape[0]):
    N = 21
    stride = 20
    plt.figure(figsize=(10, 10))
    plt.imshow(frames[t], cmap="gray")
    for y in range(N, frames[t].shape[0] - N, stride):
        for x in range(N, frames[t].shape[1] - N, stride):
            Vx_region = get_region_around(x, y, N, Vx[t])
            Vy_region = get_region_around(x, y, N, Vy[t])
            Vt_region = get_region_around(x, y, N, Vt[t])

            A = np.hstack((Vx_region.reshape(-1, 1), Vy_region.reshape(-1, 1)))
            b = -Vt_region.reshape(-1, 1)
            flow_vector, _, _, _ = np.linalg.lstsq(A, b)
            scaled_flow_vector = flow_vector.flatten()
            if (
                np.linalg.norm(scaled_flow_vector) < 1
            ): # Only plot significant flow vectors
                continue
            plt.quiver(
                x,
                y,
                scaled_flow_vector[0],
                scaled_flow_vector[1],
                color="red",
                angles="xy",
                scale_units="xy",
                scale=1,
            )
    plt.title(f"Estimated Flow Vectors (Frame {t + 1})")
    plt.axis("off")
    plt.show()

# %% [markdown]
# ### Fast Lukas-Kanade solution

# %%
# super fast lucas-kanade
import numpy as np
import scipy


def get_volume_gradients(
    V: np.ndarray, xy_sigma: float, t_sigma: float, xy_radius: int, t_radius: int
) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """
    Compute the spatial and temporal gradients of the input volume V.
    V: input volume (t, x, y)
    Returns: Vx, Vy, Vt
    """
    Vx, Vy, Vt = [
```

```
        scipy.ndimage.gaussian_filter(
            V,
            sigma=(t_sigma, xy_sigma, xy_sigma),
            order=order,
            radius=(t_radius, xy_radius, xy_radius),
        )
        for order in [[0, 0, 1], [0, 1, 0], [1, 0, 0]]
    ]
    return Vx, Vy, Vt


def get_flow_vector_lukas_kanade(
    Vx: np.ndarray, Vy: np.ndarray, Vt: np.ndarray, xy_radius: int, t_radius: int
) -> tuple[np.ndarray, np.ndarray]:
    """
    Vx, Vy, Vt: volumes (t, x, y)
    xy_radius: spatial radius for uniform filter
    t_radius: temporal radius for uniform filter
    """

    Vx2 = Vx * Vx
    Vy2 = Vy * Vy
    VxVy = Vx * Vy
    VxVt = Vx * Vt
    VyVt = Vy * Vt
    Sxx, Syy, Sxy, Sxt, Syt = [
        scipy.ndimage.uniform_filter(arr, size=(t_radius, xy_radius, xy_radius))
        for arr in [Vx2, Vy2, VxVy, VxVt, VyVt]
    ]

    det = Sxx * Syy - Sxy * Sxy
    u = -(Sxt * Syy - Syt * Sxy) / det
    v = -(Sxx * Syt - Sxy * Sxt) / det

    return u, v


# %%
# again, go through all frames and plot the flow vectors, with stride of 20

for t in range(frames.shape[0]):
    N = 21
    stride = 20
    plt.figure(figsize=(10, 10))
    plt.imshow(frames[t], cmap="gray")
    for y in range(N, frames[t].shape[0] - N, stride):
        for x in range(N, frames[t].shape[1] - N, stride):
            flow_vector = np.array([u[t, y, x], v[t, y, x]]) * 10
            if np.linalg.norm(flow_vector) < 1: # Only plot significant flow vectors
                continue
            plt.quiver(
                x,
                y,
                flow_vector[0],
                flow_vector[1],
                color="red",
                angles="xy",
                scale_units="xy",
                scale=1,
```

```
            )
    plt.title(f"Estimated Flow Vectors (Frame {t + 1})")
    plt.axis("off")
    plt.show()

# %% [markdown]
# ### Experiments on my data

# %% [markdown]
# #### Aperture Problem

# %%
from PIL.Image import new
import numpy as np
import av

container = av.open("./videos/apenture.mp4")
downscaled_shape = (640, 360)
i = 0
volume = []
for frame in container.decode(video=0):
    img = frame.to_image() # PIL/Pillow image
    # arr = np.asarray(img) # numpy array
    # aspect ratio is 16:9, so downscale to 160x90
    # downscale everything to 640x360
    # downscale everything to
    new_img = img.resize(downscaled_shape)
    arr = np.asarray(new_img)
    arr_gy = arr.mean(axis=2, dtype=np.float32) / 255.0 # cropp a little
    arr_gy = np.fliplr(arr_gy.T) # ensure it is as recorderd
    volume.append(arr_gy)
volume = np.array(volume)


# %%
# post-process
volume = volume[:, :500, :] # remove irrelevant right one
volume = volume[50:] # drop first 50 frames as camera was sharpening


# %%
# show the 50th frame
plt.figure()
plt.imshow(volume[200], cmap="gray")
plt.title("Frame 50")
# plt.axis("off")
plt.show()

# %%
Vx, Vy, Vt = get_volume_gradients(
    volume, xy_sigma=1, t_sigma=1, xy_radius=5, t_radius=5
)

u, v = get_flow_vector_lukas_kanade(Vx, Vy, Vt, xy_radius=5, t_radius=5)
#

# Do something!

# %%
```

```
### Apply optical flow - again,
# go through all frames and plot the flow vectors, with stride of 20


fig, ax = plt.subplots()
for t in range(0, volume.shape[0], 10):
    ax.clear()
    stride = 20
    ax.imshow(volume[t], cmap="gray")
    for y in range(stride, volume[t].shape[0] - stride + 1, stride):
        for x in range(stride, volume[t].shape[1] - stride + 1, stride):
            flow_vector = np.array([u[t, y, x], v[t, y, x]]) * 10
            if np.linalg.norm(flow_vector) < 1: # Only plot significant flow vectors
                continue
            ax.quiver(
                x,
                y,
                flow_vector[0],
                flow_vector[1],
                color="red",
                angles="xy",
                scale_units="xy",
                scale=1,
            )

    ax.set_title(f"Estimated Flow Vectors (Frame {t + 1})")
    ax.axis("off")
    plt.pause(1)
    # plt.show()

# %%


# %% [markdown]
# #### Rolling ball problem

# %%
from PIL.Image import new
import numpy as np
import av

container = av.open("./videos/normal.mp4")
downscaled_shape = (640, 360)
i = 0
volume = []
for frame in container.decode(video=0):
    img = frame.to_image() # PIL/Pillow image
    # arr = np.asarray(img) # numpy array
    # aspect ratio is 16:9, so downscale to 160x90
    # downscale everything to 640x360
    # downscale everything to
    new_img = img.resize(downscaled_shape)
    arr = np.asarray(new_img)
    arr_gy = arr.mean(axis=2, dtype=np.float32) / 255.0 # cropp a little
    arr_gy = np.fliplr(arr_gy.T) # ensure it is as recorderd
    volume.append(arr_gy)
volume = np.array(volume)


```

```
# %%
# post-process
volume = volume[:, :600, 50:250] # remove irrelevant right one


# %%
# show the 50th frame
plt.figure()
plt.imshow(volume[0], cmap="gray")
plt.title("Frame 50")
# plt.axis("off")
plt.show()

# %%
Vx, Vy, Vt = get_volume_gradients(
    volume, xy_sigma=1, t_sigma=1, xy_radius=5, t_radius=5
)

u, v = get_flow_vector_lukas_kanade(Vx, Vy, Vt, xy_radius=5, t_radius=5)
#

# Do something!

# %%
### Apply optical flow - again,
# go through all frames and plot the flow vectors, with stride of 20

for t in range(0, volume.shape[0], 10):
    stride = 20
    plt.imshow(volume[t], cmap="gray")
    for y in range(stride, volume[t].shape[0] - stride + 1, stride):
        for x in range(stride, volume[t].shape[1] - stride + 1, stride):
            flow_vector = np.array([u[t, y, x], v[t, y, x]]) * 10
            if np.linalg.norm(flow_vector) < 1: # Only plot significant flow vectors
                continue
            plt.quiver(
                x,
                y,
                flow_vector[0],
                flow_vector[1],
                color="red",
                angles="xy",
                scale_units="xy",
                scale=1,
            )
    plt.title(f"Estimated Flow Vectors (Frame {t + 1})")
    plt.axis("off")
    plt.show()

# %% [markdown]
# #### Moving camera, but not the ball

# %%
from PIL.Image import new
import numpy as np
import av
import matplotlib.pyplot as plt

container = av.open("./videos/adverserial_one.mp4")
```

```
downscaled_shape = (640, 360)
i = 0
volume = []
for frame in container.decode(video=0):
    img = frame.to_image() # PIL/Pillow image
    # arr = np.asarray(img) # numpy array
    # aspect ratio is 16:9, so downscale to 160x90
    # downscale everything to 640x360
    # downscale everything to
    new_img = img.resize(downscaled_shape)
    arr = np.asarray(new_img)
    arr_gy = arr.mean(axis=2, dtype=np.float32) / 255.0 # cropp a little
    arr_gy = np.fliplr(arr_gy.T) # ensure it is as recorderd
    volume.append(arr_gy)
volume = np.array(volume)


# %%
# post-process
volume = volume[:, :, 20:] # remove irrelevant right one


# %%
# show the 50th frame
plt.figure()
plt.imshow(volume[300], cmap="gray")
plt.title("Frame 50")
plt.axis("off")
plt.show()

# %%
Vx, Vy, Vt = get_volume_gradients(
    volume, xy_sigma=1, t_sigma=1, xy_radius=5, t_radius=5
)

u, v = get_flow_vector_lukas_kanade(Vx, Vy, Vt, xy_radius=5, t_radius=5)
#

# Do something!

# %%
### Apply optical flow - again,
# go through all frames and plot the flow vectors, with stride of 20

for t in range(0, volume.shape[0], 10):
    stride = 20
    plt.imshow(volume[t], cmap="gray")
    for y in range(stride, volume[t].shape[0] - stride + 1, stride):
        for x in range(stride, volume[t].shape[1] - stride + 1, stride):
            flow_vector = np.array([u[t, y, x], v[t, y, x]])
            if np.linalg.norm(flow_vector) < 1: # Only plot significant flow vectors
                continue
            plt.quiver(
                x,
                y,
                flow_vector[0],
                flow_vector[1],
                color="red",
                angles="xy",
```

```
                scale_units="xy",
                scale=1 / 10,
            )
    plt.title(f"Estimated Flow Vectors (Frame {t + 1})")
    plt.axis("off")
    plt.show()
```